

Boot-time context-store seed

What the client writes into the **context-store** at boot, and how it feeds the schema-driven **x-context-source** wiring.

How it works

At app start (before the first backend request and before the **initialization** workflows run), a **context provider** seeds the values the client already holds into the context-store. A backend schema can then declare **x-context-source: { context: { boundary, key } }** on an input property, and the generic client resolves it from these slots — **no per-flow client code**.

- Writer: `examples/vue-app/src/boot/contextProviders.ts` (`webAmbientProvider`)
- Runner: `@burgan-tech/app-host runContextProviders` (eager, at boot)
- Reader: `pseudo-ui / workflow-manager x-context-source` resolver
- Vocabulary (keys): `examples/vue-app/src/boot/constants.ts` (CTX)

Every slot has a **boundary** (the lifecycle/scope it belongs to) and a **storage** tier. **x-context-source** references both the boundary and the key. Example values below are from a real boot (web / headless Chrome).

Boundaries

Boundary	Meaning
device	The device/browser (pre-login facts).
subject	The customer the operation is for = scope .
user	The actor — the user acting on the customer's behalf.

(For an individual customer **subject** and **user** coincide; they are kept distinct.)

Ambient values (seeded by the context provider)

All of these are in the **device** boundary, **memory** storage.

CTX key	context-store key	Boundary	Storage	Meaning	Example value
clientId	app.clientId	device	memory	Client id (the single bootstrap constant)	IbWeb
appVersion	app.version	device	memory	Application version	0.1.0
deviceId	device.id	device	memory	Device id (persistent id, seeded to memory)	b28a45a1-c9c5-48dc-bc2b-558a0636a48c
installationId	device.installation.id	device	memory	Installation id (new per app open)	33a2a899-0a82-4650-8933-1f77456002a8
osName	device.osName	device	memory	Operating system	macOS
osVersion	device.osVersion	device	memory	OS version (navigator)	5.0 (Macintosh; Intel Mac OS X 10_15_7)...
deviceModel	device.model	device	memory	Device model	browser
manufacturer	device.manufacturer	device	memory	Manufacturer (navigator.vendor)	Google Inc.
screenResolution	device.screenResolution	device	memory	Screen resolution	1280x720

CTX key	context-store key	Boundary	Storage	Meaning	Example value
language	device.language	device	memory	Device language	en-US
timezone	device.timezone	device	memory	Time zone	Europe/Istanbul
userAgent	device.userAgent	device	memory	Full user agent	Mozilla/5.0 ... Safari/537.36

Platform note: these come from the **web** adapter (`navigator/screen`). A native/Dart adapter supplies its own provider reporting the real OS/device; the keys (contract) stay the same.

Other values written at boot (not by the provider)

context-store key	Boundary	Storage	Meaning	Written by
app.locale	device	memory	Active UI locale	<code>boot/main.ts</code> (from router; updated on locale change)
app.idmBase	device	memory	IDM host base URL	<code>boot/main.ts</code> (from environment <code>hosts</code> config)
device.registration	device	localStorage	device-manager registration result	<code>provisionDevice</code>

Written after login (for reference — not boot)

identity	Boundary	Meaning	Source
activeSubject	subject	customer / scope	<code>boot/login.ts</code> — JWT <code>sub</code>
activeUser	user	actor / the acting user	<code>boot/login.ts</code> — JWT <code>act_sub</code> ?? <code>sub</code>

Identity model: `subject` = customer (= "scope"), `actor` = the user acting on the customer's behalf. For an individual customer the two coincide; both are kept distinct in the context-store.